

sudoku

October 6, 2026

1 Trabalho Prático — Sudoku Genérico como CSP (CP-SAT)

Este notebook contém: - justificação da técnica e da abstração escolhida, - classe genérica `Box` (grupo de células “todos diferentes”, algumas fixas), - especializações `Cube` (bloco $n \times n$) e `Path` (troço reto), - geração aleatória de pistas (devolve um `Box`), - modelo CSP (OR-Tools CP-SAT) que aceita um número arbitrário de grupos, - validação automática e testes ($n=3$ e $n=2$), - extensões: X-Sudoku, Windoku e escala ($n=6$).

Instalação:

1.0.1 Decisões de modelação (justificação)

- **Técnica:** CP-SAT do OR-Tools. O Sudoku é um CSP com restrições `AllDifferent`, que o CP-SAT suporta nativamente com boa propagação.
- **Abstração:** o modelo só conhece “grupos de células com restrição todos diferentes e algumas células fixas”. Linhas, colunas, blocos, pistas aleatórias, diagonais e regiões extra são todos `Box` (ou subclasses), por isso o modelo nunca distingue a origem de um grupo.
- **Box:** guarda um dicionário `(linha, coluna) -> valor | None`. Não sabe nada sobre linhas, blocos ou Sudoku, apenas valida coordenadas e valores em $[1, n^2]$.
- **Pistas aleatórias:** células distintas com valores aleatórios. Podem tornar o puzzle impossível (dois valores iguais na mesma linha, por exemplo). Por isso `gerar_e_resolver` tenta novas pistas até obter um puzzle solúvel, com um número máximo de tentativas. Assim obtemos sempre um exemplo útil, e o insucesso fica reportado se esgotar as tentativas.
- **Sem solução:** o método `resolver` devolve `None` e guarda o estado do solver (`INFEASIBLE`), de forma distinguível de uma grelha válida.

```
[ ]: ## 1. Importações e funções utilitárias
import random
import time
from ortools.sat.python import cp_model
```

1.1 2. Box — grupo genérico de células (R1)

Guarda `(linha, coluna) -> valor` ou `None`. `n` define a grelha $n^2 \times n^2$. - `add(i, j, val=None)` levanta `ValueError` para coordenadas fora da grelha ou valores fora de $[1, n^2]$. - `matriz()` devolve a matriz $n^2 \times n^2$ com zeros nas células não pertencentes ao grupo ou não fixas.

```
[ ]: class Box:
    def __init__(self, n, celulas=None):
```

```

self.n = n
self.N = n * n
self.celulas = {}
if celulas:
    for (i, j), val in dict(celulas).items():
        self.add(i, j, val)

def add(self, i, j, val=None):
    if not (0 <= i < self.N and 0 <= j < self.N):
        raise ValueError(f"Coordenadas ({i},{j}) fora da grelha {self.
↪N}x{self.N}")
    if val is not None and not (1 <= val <= self.N):
        raise ValueError(f"Valor {val} fora do intervalo [1,{self.N}]")
    if val is not None or (i, j) not in self.celulas:
        self.celulas[(i, j)] = val
    return self

def matriz(self):
    m = [[0] * self.N for _ in range(self.N)]
    for (i, j), val in self.celulas.items():
        if val is not None:
            m[i][j] = val
    return m

def __len__(self):
    return len(self.celulas)

def __repr__(self):
    return f"{type(self).__name__}({len(self)} células, {sum(v is not None_
↪for v in self.celulas.values())} fixas)"

```

1.2 3. Cube e Path — duas formas concretas de grupo (R2, R3)

- Cube(n, i, j): bloco $n \times n$ com canto superior esquerdo em $(i \cdot n, j \cdot n)$, com $0 \leq i, j < n$.
- Path(n, inicio, fim): troço reto (horizontal ou vertical), inclusive, em qualquer sentido.

```

[ ]: class Cube(Box):
    def __init__(self, n, i, j):
        super().__init__(n)
        if not (0 <= i < n and 0 <= j < n):
            raise ValueError(f"Índices de bloco ({i},{j}) fora de [0,{n - 1}]")
        self.i, self.j = i, j
        for di in range(n):
            for dj in range(n):
                self.add(i * n + di, j * n + dj)

```

```

class Path(Box):
    def __init__(self, n, inicio, fim):
        super().__init__(n)
        (i0, j0), (i1, j1) = inicio, fim
        if i0 != i1 and j0 != j1:
            raise ValueError("Path tem de ser horizontal ou vertical")
        passo_i = (i1 > i0) - (i1 < i0)
        passo_j = (j1 > j0) - (j1 < j0)
        comprimento = max(abs(i1 - i0), abs(j1 - j0)) + 1
        for t in range(comprimento):
            self.add(i0 + t * passo_i, j0 + t * passo_j)

```

1.3 4. Geração aleatória de pistas (R4)

Devolve um Box com k células distintas, cada uma fixa a um valor aleatório em $[1, n^2]$. Por omissão $k = n$. Não é preciso nenhuma classe nova.

```

[ ]: def pistas_aleatorias(n, k=None, rng=None):
    rng = rng or random
    N = n * n
    k = n if k is None else k
    if k > N * N:
        raise ValueError("k maior do que o número de células")
    caixa = Box(n)
    todas = [(i, j) for i in range(N) for j in range(N)]
    for (i, j) in rng.sample(todas, k):
        caixa.add(i, j, rng.randint(1, N))
    return caixa

```

1.4 5. Modelo CSP e resolução (R5)

SudokuCSP(n) cria uma variável inteira por célula em $[1, n^2]$. - `adicionar(*grupos)`: para cada grupo impõe AllDifferent nas suas células e fixa as que têm valor. Não distingue a origem do grupo. - `resolver()`: devolve a grelha (lista de listas) ou None se não há solução. O estado fica em `self.estado`.

```

[ ]: class SudokuCSP:
    def __init__(self, n):
        self.n = n
        self.N = n * n
        self.m = cp_model.CpModel()
        self.v = {(i, j): self.m.NewIntVar(1, self.N, f"c_{i}_{j}")
                    for i in range(self.N) for j in range(self.N)}
        self.estado = None
        self.tempo = None

    def adicionar(self, *grupos):

```

```

    for g in grupos:
        if g.n != self.n:
            raise ValueError("Grupo com n diferente do modelo")
        if len(g.celulas) > 1:
            self.m.AddAllDifferent([self.v[c] for c in g.celulas])
        for c, val in g.celulas.items():
            if val is not None:
                self.m.Add(self.v[c] == val)
    return self

def resolver(self, tempo_max=60.0, workers=8):
    solver = cp_model.CpSolver()
    solver.parameters.max_time_in_seconds = tempo_max
    solver.parameters.num_workers = workers
    t0 = time.perf_counter()
    st = solver.Solve(self.m)
    self.tempo = time.perf_counter() - t0
    self.estado = solver.StatusName(st)
    if st in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return [[solver.Value(self.v[(i, j)]) for j in range(self.N)] for i
↪in range(self.N)]
    return None

```

1.5 6. Montagem do Sudoku completo (R6)

Linhas + colunas + blocos + pistas aleatórias. As linhas e colunas são `Path`, os blocos são `Cube`. `gerar_e_resolver` repete com novas pistas enquanto o puzzle for impossível.

```

[ ]: def linhas_colunas_blocos(n):
    N = n * n
    linhas = [Path(n, (i, 0), (i, N - 1)) for i in range(N)]
    colunas = [Path(n, (0, j), (N - 1, j)) for j in range(N)]
    blocos = [Cube(n, i, j) for i in range(n) for j in range(n)]
    return linhas, colunas, blocos

def montar_sudoku(n, pistas, extras=()):
    linhas, colunas, blocos = linhas_colunas_blocos(n)
    modelo = SudokuCSP(n)
    modelo.adicionar(*linhas, *colunas, *blocos, pistas, *extras)
    return modelo

def gerar_e_resolver(n, k=None, seed=None, tentativas=50, extras=(),
↪tempo_max=60.0):
    rng = random.Random(seed)
    for t in range(1, tentativas + 1):

```

```

        pistas = pistas_aleatorias(n, k, rng)
        modelo = montar_sudoku(n, pistas, extras)
        grelha = modelo.resolver(tempo_max=tempo_max)
        if grelha is not None:
            return grelha, pistas, t, modelo
    return None, None, tentativas, None

def mostrar_grelha(grelha, n, pistas=None):
    N = n * n
    larg = len(str(N))
    linhas = []
    for i in range(N):
        if i % n == 0 and i > 0:
            linhas.append("-" * ((larg + 1) * N + n - 1))
        partes = []
        for j in range(N):
            if j % n == 0 and j > 0:
                partes.append("|")
            fixa = pistas is not None and pistas.celulas.get((i, j)) is not None
            partes.append(str(grelha[i][j]).rjust(larg) + ("*" if fixa else " ")
↪))
        linhas.append("".join(partes))
    print("\n".join(linhas))
    if pistas is not None:
        print("( * = pista fixa)")

```

1.6 7. Validação automática

Verifica numa grelha resolvida: - cada linha, coluna e bloco contém exatamente $1..n^2$, - as pistas mantêm o valor, - add rejeita coordenadas e valores inválidos.

```

[ ]: def validar_solucão(grelha, n, pistas=None):
    N = n * n
    alvo = set(range(1, N + 1))
    erros = []
    for i in range(N):
        if set(grelha[i]) != alvo:
            erros.append(f"Linha {i} inválida")
    for j in range(N):
        if {grelha[i][j] for i in range(N)} != alvo:
            erros.append(f"Coluna {j} inválida")
    for bi in range(n):
        for bj in range(n):
            vals = {grelha[bi * n + a][bj * n + b] for a in range(n) for b in
↪range(n)}
            if vals != alvo:

```

```

        erros.append(f"Bloco ({bi},{bj}) inválido")
    if pistas is not None:
        for (i, j), val in pistas.celulas.items():
            if val is not None and grelha[i][j] != val:
                erros.append(f"Pista ({i},{j})={val} não respeitada (obtido_
↪{grelha[i][j]})")
    return erros

def testar_add(n):
    N = n * n
    b = Box(n)
    casos_invalidos = [(-1, 0, None), (0, -1, None), (N, 0, None), (0, N, None),
                        (0, 0, 0), (0, 0, N + 1)]
    for (i, j, v) in casos_invalidos:
        try:
            b.add(i, j, v)
        except ValueError:
            continue
        raise AssertionError(f"add({i},{j},{v}) devia ter falhado")
    b.add(0, 0, 1)
    b.add(N - 1, N - 1, N)
    assert b.matriz()[0][0] == 1 and b.matriz()[N - 1][N - 1] == N
    return True

def testar_paths_cubes(n):
    N = n * n
    assert set(Path(n, (0, 0), (0, N - 1)).celulas) == {(0, j) for j in_
↪range(N)}
    assert set(Path(n, (0, N - 1), (0, 0)).celulas) == {(0, j) for j in_
↪range(N)} # sentido inverso
    assert set(Path(n, (N - 1, 2), (0, 2)).celulas) == {(i, 2) for i in_
↪range(N)} # vertical, inverso
    assert set(Cube(n, 1, 0).celulas) == {(n + a, b) for a in range(n) for b in_
↪range(n)}
    try:
        Path(n, (0, 0), (1, 1))
        raise AssertionError("Path diagonal devia falhar")
    except ValueError:
        pass
    return True

print("add rejeita inválidos:", testar_add(3), testar_add(2))
print("Path/Cube corretos:", testar_paths_cubes(3), testar_paths_cubes(2))

```

1.7 8. Fluxo completo com $n = 3$ (9×9) e $n = 2$ (4×4)

```
[ ]: for n in (3, 2):
    grelha, pistas, tent, modelo = gerar_e_resolver(n, seed=42)
    print(f"\n=== n = {n} ({n*n}x{n*n}) | tentativas: {tent} | estado: {modelo.
    estado} "
          f"| tempo: {modelo.tempo:.3f}s ===")
    print("Pistas:", pistas.celulas)
    mostrar_grelha(grelha, n, pistas)
    erros = validar_solucao(grelha, n, pistas)
    print("Erros de validação:", erros)
    assert not erros
```

1.7.1 Puzzle sem solução (sinalização distinguível)

Duas pistas com o mesmo valor na mesma linha tornam o puzzle impossível: `resolver()` devolve `None` e o estado é `INFEASIBLE`.

```
[ ]: n = 3
    impossivel = Box(n).add(0, 0, 5).add(0, 1, 5)
    modelo = montar_sudoku(n, impossivel)
    res = modelo.resolver()
    print("Resultado:", res, "| estado:", modelo.estado)
    assert res is None and modelo.estado == "INFEASIBLE"
```

1.8 9. (Bónus) X-Sudoku e Windoku

Basta acrescentar **novos grupos** (`Box`); o modelo não muda. - X-Sudoku: as duas diagonais principais. - Windoku ($n = 3$): 4 blocos 3×3 extra com cantos em (1,1), (1,5), (5,1), (5,5).

```
[ ]: def diagonais(n):
    N = n * n
    d1, d2 = Box(n), Box(n)
    for i in range(N):
        d1.add(i, i)
        d2.add(i, N - 1 - i)
    return [d1, d2]

def windoku(n=3):
    assert n == 3, "Windoku definido aqui para n = 3"
    extras = []
    for (r, c) in [(1, 1), (1, 5), (5, 1), (5, 5)]:
        b = Box(n)
        for a in range(3):
            for d in range(3):
                b.add(r + a, c + d)
        extras.append(b)
```

```

    return extras

# X-Sudoku
n = 3
grelha, pistas, tent, modelo = gerar_e_resolver(n, seed=7, extras=diagonais(n))
mostrar_grelha(grelha, n, pistas)
alvo = set(range(1, 10))
assert not validar_solucao(grelha, n, pistas)
assert {grelha[i][i] for i in range(9)} == alvo and {grelha[i][8 - i] for i in
↪range(9)} == alvo
print("X-Sudoku OK | estado:", modelo.estado)

# Windoku
grelha, pistas, tent, modelo = gerar_e_resolver(3, seed=3, extras=windoku(3))
assert not validar_solucao(grelha, 3, pistas)
for (r, c) in [(1, 1), (1, 5), (5, 1), (5, 5)]:
    assert {grelha[r + a][c + b] for a in range(3) for b in range(3)} == alvo
print("Windoku OK | estado:", modelo.estado)

```

1.9 10. (Bónus) Escala: $n = 6$ (36×36)

O mesmo código, sem alterações. O tempo depende das pistas aleatórias (pouco restritivas, o que dá muitas soluções).

```

[ ]: for n in (3, 4, 5, 6):
    t0 = time.perf_counter()
    grelha, pistas, tent, modelo = gerar_e_resolver(n, seed=1, tempo_max=120)
    total = time.perf_counter() - t0
    if grelha is None:
        print(f"n={n}: sem solução encontrada")
        continue
    erros = validar_solucao(grelha, n, pistas)
    print(f"n={n} ({n*n}x{n*n}): estado={modelo.estado}, solver={modelo.tempo:.
↪2f}s, "
        f"total={total:.2f}s, tentativas={tent}, válido={not erros}")

```

1.10 11. Conclusões e discussão

- O modelo só conhece grupos “todos diferentes” com células fixas: linhas, colunas, blocos, pistas, diagonais e regiões do Windoku são todos Box e passam pelo mesmo método adicionar.
- Box valida coordenadas e valores (R1); Cube e Path são especializações (R2, R3); as pistas aleatórias devolvem um Box (R4); o modelo CSP aceita um número arbitrário de grupos (R5); o Sudoku completo junta linhas, colunas, blocos e pistas (R6).
- Um puzzle com pistas aleatórias pode ser impossível. Optou-se por tentar novas pistas até haver solução, porque queremos sempre um exemplo resolvido; o caso impossível é sinalizado por None e INFEASIBLE.

- Limites: o número de variáveis cresce com n e o de restrições `AllDifferent` com $3 \cdot n^2$. Para $n = 6$ o tempo aumenta mas continua viável com poucas pistas.
- Extensões possíveis: Sudoku jigsaw (um `Box` por região irregular, construído célula a célula) e Sudoku 3D (grupos em $n^2 \times n^2 \times n^2$, generalizando as coordenadas para 3 dimensões).

1.10.1 Como reproduzir

1. `pip install ortools`
2. Executar todas as células por ordem.
3. Exportar para PDF: `jupyter nbconvert --to pdf sudoku.ipynb`